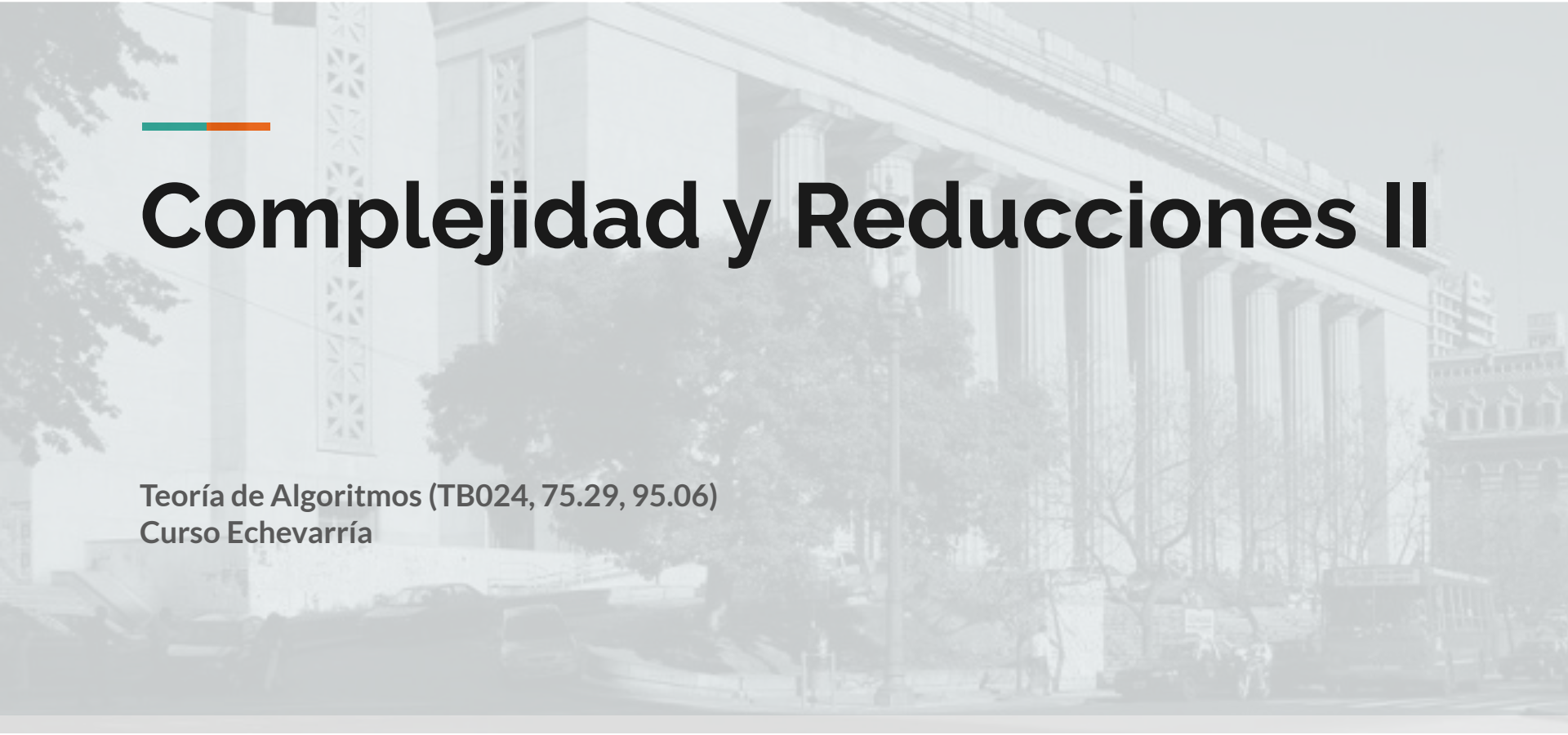


Complejidad y Reducciones II

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Complejidad y Reducciones: Repaso de la clase anterior



Complejidad y Reducciones: Repaso

- La máquina **determinística** de Turing es una máquina teórica que puede resolver lo mismo que cualquier computadora.
- La máquina **no determinística** puede replicarse las veces que haga falta y procesar en paralelo. (No existe una máquina que pueda hacer eso en la realidad)
- A cada problema de optimización le corresponde un **problema de decisión,** que se responde por sí o por no.
- Las **reducciones** permiten “convertir” un problema en otro distinto. Son útiles si sé resolver el segundo problema pero no el primero.
- Las más interesantes son las que agregan menos “overhead” al problema (las **polinomiales**).

Reducciones II



Créditos: Débora Bonenberg

Reducciones: Ejemplo 6

Problema Conjunto Independiente (Independent Set):

- En un grafo $G = (V,E)$, un conjunto independiente es un conjunto de vértices que no comparte ninguna arista entre sí.
- El problema consiste en encontrar el conjunto independiente más grande posible.

Dado un grafo G y un número k,

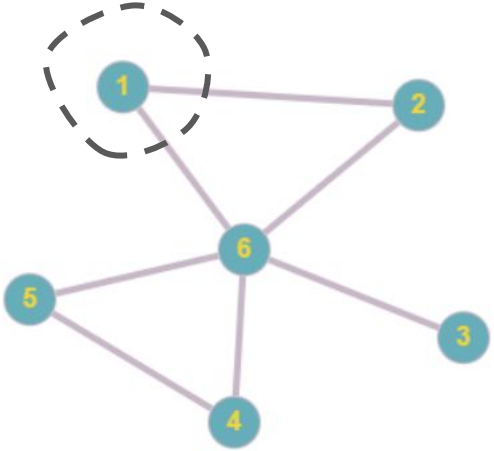
¿Existe en G un conjunto independiente de un tamaño de por lo menos k?

Reducciones: Ejemplo 6

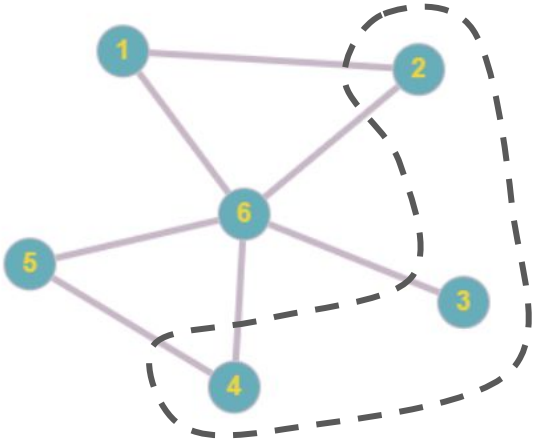
Problema Conjunto Independiente (Independent Set):

Dado un grafo G y un número k,

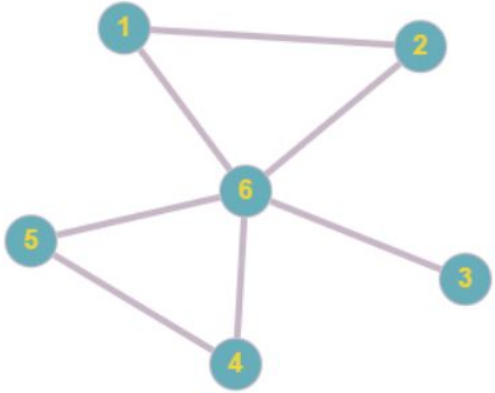
¿Existe en G un conjunto independiente de un tamaño de por lo menos k?



k=1



k=3



k=4

Reducciones: Ejemplo 6

Problema Cobertura de Vértices (Vertex Cover):

- En un grafo $G = (V,E)$, un subconjunto S de vértices cubre al grafo si todas las aristas del grafo tienen al menos un extremo en S.
- El problema consiste en encontrar el subconjunto más chico posible.

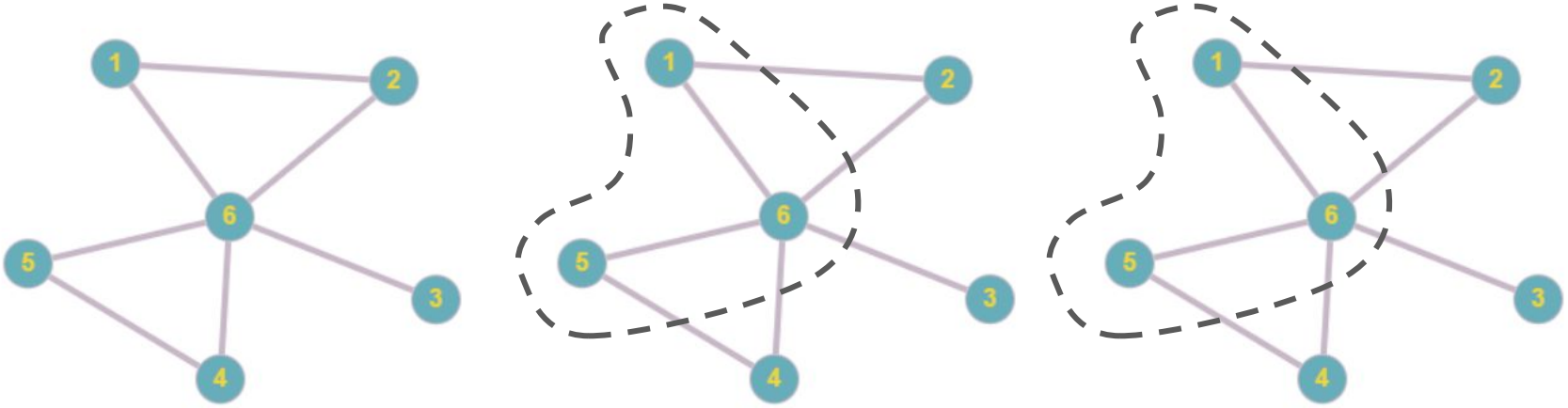
Dado un grafo G y un número k,
¿Existe en G un Vertex Cover de un tamaño de a lo sumo k?

Reducciones: Ejemplo 6

Problema Cobertura de Vértices (Vertex Cover):

Dado un grafo G y un número k,

¿Existe en G un Vertex Cover de un tamaño de a lo sumo k?



k=1 k=3 k=4

Reducciones: Ejemplo 6

Independent Set $\leq_P$ Vertex Cover:

- Sea V el conjunto de vértices del grafo y S un Independent Set.
- Para cualquier arista e, no puede tener ambos extremos en S.
- Todas las aristas tienen al menos un vértice en S y otro en $(V - S)$.
  $\therefore$ $(V - S)$ es un Vertex Cover.

Haciendo el camino inverso, se puede demostrar que:

Vertex Cover $\leq_P$ *Independent Set*

Reducciones: Ejemplo 6

Independent Set $\leq_P$ Vertex Cover



Grafo inicial

- No hace falta Transformación

Resolver el Problema VC

Los nodos que no forman parte del VC, forman parte del IS

Solución de Independent Set

Reducciones: Ejemplo 6

Vertex Cover $\leq_P$ Independent Set



Grafo inicial

- No hace falta Transformación

Resolver el Problema IS

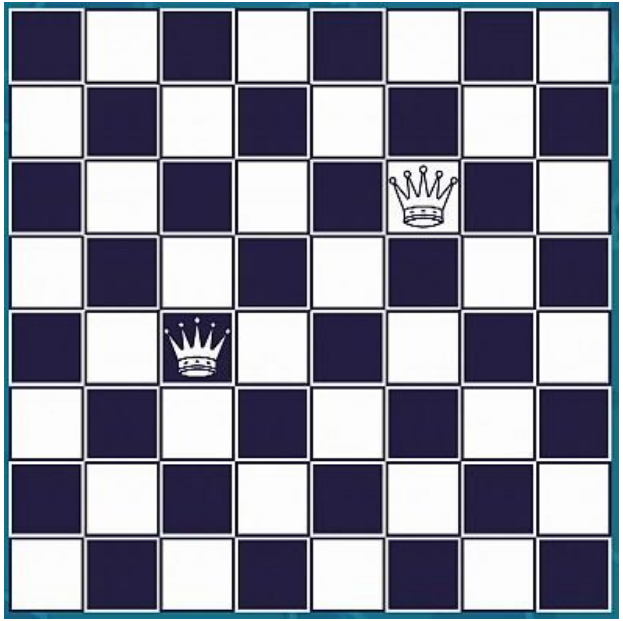
Los nodos que no forman parte del IS, forman parte del VC

Solución de Vertex Cover

Reducciones: Ejemplo 7

Problema de las 8 reinas:

- En un tablero de ajedrez, colocar 8 reinas de manera que ninguna amenace a otra.
- O sea, que no haya 2 reinas en la misma fila, columna o diagonal.
- Se puede generalizar a N reinas en un tablero de NxN.
- ¿A qué problema se puede parecer?



Reducciones: Ejemplo 7

Problema de las N reinas:

- Puede plantearse como un Independent Set
- Cada casillero del tablero es un vértice.
- Agrego aristas que lo vinculen con aquellos vértices que están en la misma fila, columna o diagonal.

- El problema es encontrar N vértices no vinculados entre sí ⇒ Independent Set.

Reducciones: Ejemplo 7

N – Reinas $\leq_P$ Independent Set

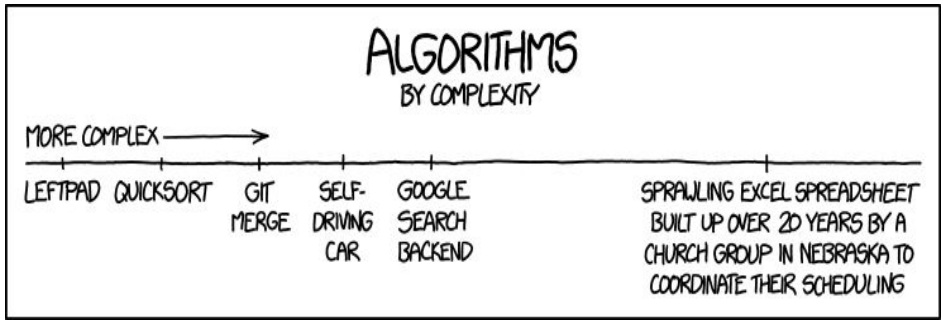
NR	Transformación Entrada	IS	Transform. Salida	NR
Tablero inicial	• Definir los nodos correspondientes a cada casilla del tablero • Definir las aristas correspondientes a las casillas en la misma fila, columna o diagonal	Resolver el Problema IS para k = N	Los nodos que forman parte del IS son las casillas en donde colocar las reinas	Solución de N reinas

Reducciones: Ejemplo 7

¿Independent Set $\leq_P N$ – Reinas ?
Al revés no funciona:

- No todos los problemas de IS pueden graficarse como un tablero de ajedrez.
  - Por ejemplo, un grafo (trivial) en el que todos los nodos se vinculan con todos.
  - O uno en el que tengo dos conjuntos disjuntos no vinculados entre sí.
- Sí puedo decir que el problema de N-Reinas no es más difícil que el IS.
- En sentido contrario, no puedo decir nada (Posiblemente, IS sea más difícil) .

Complejidad



https://imgs.xkcd.com/comics/algorithms.png

Complejidad

- Instancia es cada una de las posibles entradas.
- La complejidad se define según la relación entre la cantidad de instrucciones computadas y el tamaño de la instancia.
- Sólo puedo medirla relativamente:
  - Ej: “Si duplico el tamaño de la instancia, se cuadruplica la complejidad”
  - Ej: “Este problema es más complejo que aquél”

Clase “P”

Un problema de decisión está en $\boldsymbol{P}$ *si una máquina* ***determinística*** *de Turing puede resolverlo en tiempo polinomial.*



Clase “NP”

Un problema de decisión está en $\boldsymbol{NP}$ *si una máquina* ***no determinística*** *de Turing puede resolverlo en tiempo polinomial.*

Certificaciones

- Además del algoritmo para RESOLVER un problema, necesitamos otro algoritmo para validar o CERTIFICAR una solución a dicho problema.
- Es un algoritmo que recibe el problema, recibe una solución y debe responder si esa solución satisface el problema o no.
- Ej: Recibe el grafo de un viajante, recibe un recorrido y debe responder si dicho recorrido cumple las condiciones del problema o no.

Clase “NP” (*)

Un problema de decisión está en $\mathbf{NP}$ *si tiene un* ***algoritmo*** *que lo pueda certificar en una máquina* ***determinística*** *de Turing en tiempo polinomial.*

Clase “NP”

Un problema de decisión está en $\boldsymbol{NP}$ *si una máquina* ***no determinística*** *de Turing puede resolverlo en tiempo polinomial.*

Un problema de decisión está en $\boldsymbol{NP}$ *si tiene un* ***algoritmo*** *que lo pueda certificar en una máquina* ***determinística*** *de Turing en tiempo polinomial.*

$P \subseteq NP$

- Con cualquier definición, está claro que un problema P también es NP:
  - Si una máquina determinística lo resuelve en un tiempo T, una no determinística puede hacer lo mismo con un solo hilo de ejecución.
  - Si puedo resolverlo en un tiempo T, puedo certificar una solución resolviéndola desde el principio en el mismo tiempo.

¿P = NP?

- Para los problemas en NP no se conocen algoritmos que los resuelvan en tiempo polinomial.
- Pero no se puede probar que no existan esos algoritmos.



The Simpsons, Treehouse of Horror VI, temp 7, ep 6.

NP y Reducciones

- De todos los problemas en NP, ¿cuál sería el más difícil?
- Supongamos que tenemos 2 problemas, X e Y, ambos pertenecientes a NP:
  - Si Y puede reducirse a X ($Y \leq_P X$), entonces Y no es más difícil que X.
  - Si todos los problemas de NP pueden reducirse a X, entonces no hay problemas NP más difíciles que X.
- A ese problema X lo llamamos ***NP-Completo***



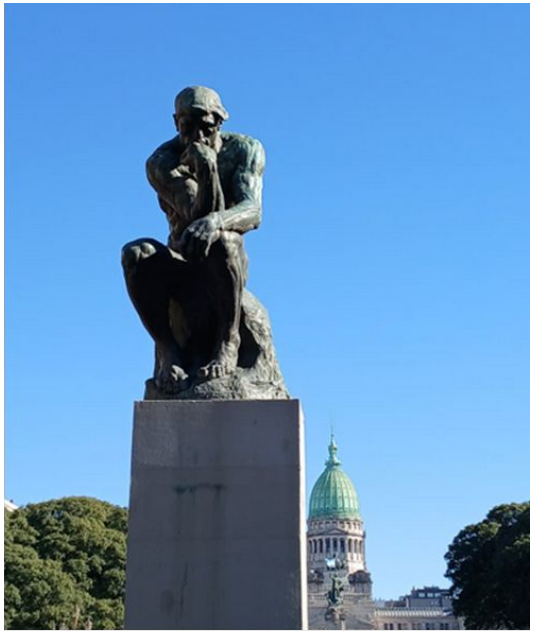
Clase “NP-Completo”

Un problema de decisión X está en NP-Completo sii todos los problemas en NP pueden reducirse a X

Clase “NP-Completo”

Dudas

- ¿Existen los problemas *NP-completos*? ¿Se puede demostrar que existen? ¿Se puede demostrar que no existen?
- ¿Existen problemas en *NP* que no sean *NP-completos*?

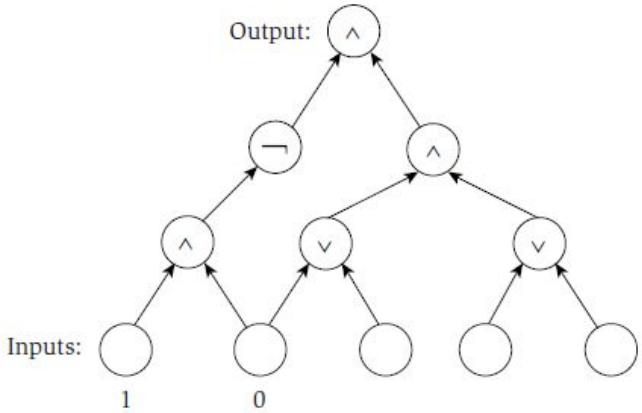


“El Pensador” - A. Rodin

Clase “NP-Completo”

Stephen Cook (1971):

- Demostró que el problema de Satisfacibilidad de Circuitos es *NP-Completo*.
- Un circuito es una serie de entradas booleanas conectadas con los operadores *AND* ($\wedge$), *OR* ($\vee$) *y NOT* ($\neg$) y una única salida, también booleana.
- El problema consiste en demostrar si para un circuito dado, hay algún set de valores para las variables de entrada que tenga como salida un 1. (sea un circuito satisfacible)
- Es un problema *NP*, y Cook demostró que cualquier problema *NP* puede reducirse a un circuito.
- O sea, que es un problema *NP-Completo*.



¿Qué valores de estas tres variables satisfacen el circuito?

Clase “NP-Completo”

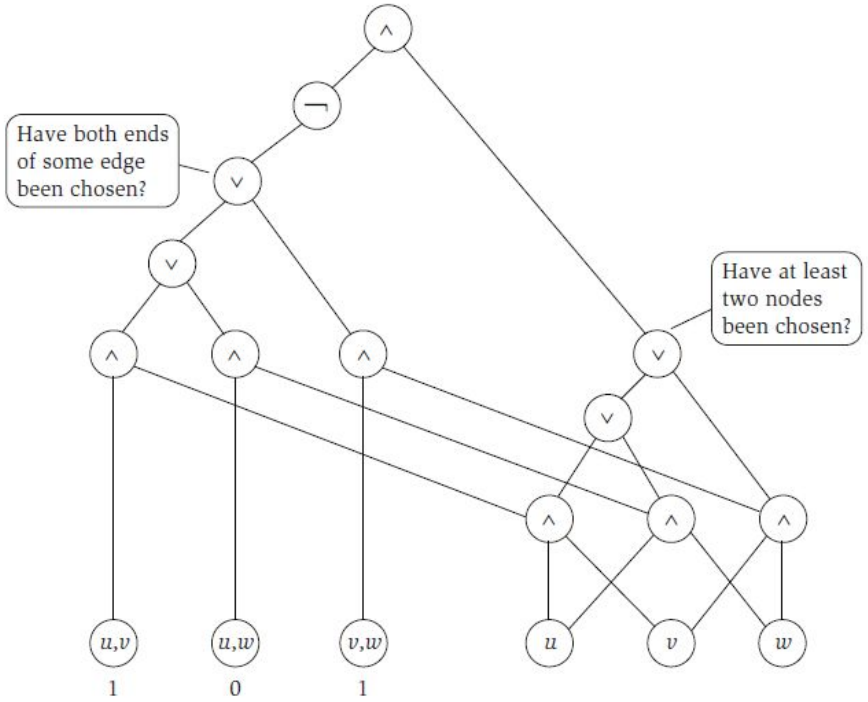


Figure 8.5 A circuit to verify whether a 3-node graph contains a 2-node independent set.

Circuito para decidir si existe un Independent Set de tamaño $k=2$ en un grafo de tres vértices $V=\{u, v, w\}$ y dos aristas $E=\{(u,v), (v,w)\}$



Clase “NP-Completo”

- A partir de esa demostración, si puedo reducir un problema Y $NP\text{-}Completo$ conocido al problema X que estoy estudiando, entonces el problema en estudio es también $NP\text{-}Completo$:
  - $Y \leq_P X \Rightarrow X$ es $NP\text{-}Completo$ (transitividad)
- Cook lo demostró para el problema SAT
- Richard Karp (1972): Creó una lista de 21 problemas $NP\text{-}Completos$.
- Ahora son miles, *literalmente*.
- Si pudiéramos resolver uno solo en tiempo polinomial, por reducción los podríamos resolver a TODOS en tiempo polinomial.

Clase “NP-Completo”

Lista de 21 problemas de Karp (1972)

- SAT (Problema de satisfacibilidad booleana, para fórmulas en forma normal conjuntiva)
  - 0-1 INTEGER PROGRAMMING (Problema de programación lineal entera)
  - CLIQUE (Problema del clique, o Problema del conjunto independiente)
    - SET PACKING (Problema del empaquetamiento de conjuntos)
    - VERTEX COVER (Problema de cobertura de vértices)
      - SET COVERING (Problema de cobertura de conjuntos)
      - FEEDBACK NODE SET (Conjunto de vértices que hay que sacar para que el grafo quede sin ciclos)
      - FEEDBACK ARC SET (Conjunto de arcos que hay que sacar para que el grafo quede DAG)
      - DIRECTED HAMILTONIAN CIRCUIT (Problema del circuito hamiltoniano dirigido)
        - UNDIRECTED HAMILTONIAN CIRCUIT (Problema del circuito hamiltoniano no dirigido)
  - 3-SAT (Problema de satisfacibilidad booleana de 3 variables por cláusula)
    - CHROMATIC NUMBER (Problema de coloreo de grafos)
      - CLIQUE COVER (Problema de la cobertura de cliques)
      - EXACT COVER (Problema de la cobertura exacta)
        - HITTING SET (formulación equivalente a Set Cover)
        - STEINER TREE (árbol generador mínimo con agregado de vértices en el grafo)
        - 3-DIMENSIONAL MATCHING (Problema del matching tridimensional)
        - KNAPSACK (Problema de la mochila)
          - JOB SEQUENCING (Problema de las secuencias de trabajo)
          - PARTITION (Problema de la partición)
            - MAX-CUT (Problema del corte máximo)

Reducciones: Ejemplo 8

Satisfacibilidad de Circuitos $\leq_P$ SAT



SC	Transformación Entrada	SAT	Transform. Salida	SC
Circuito inicial	• Transformar los nodos $x_v=\neg x_u$ en cláusulas $(x_u \vee x_v), (\neg x_u \vee \neg x_v)$ • Transformar los nodos $x_v=x_u \vee x_w$ en cláusulas $(x_v \vee \neg x_u), (x_v \vee \neg x_w)$ y $(\neg x_v \vee x_u \vee x_w)$ • Transformar los nodos $x_v=x_u \wedge x_w$ en cláusulas $(\neg x_v \vee x_u), (\neg x_v \vee x_w), (x_v \vee \neg x_u \vee \neg x_w)$ • Transformar las constantes en cláusulas (x) o $(\neg x)$	Resolver el Problema SAT	No hace falta transformación	Solución de Satisfacibilidad de Circuitos

$\therefore$ SAT es *NP-Completo*

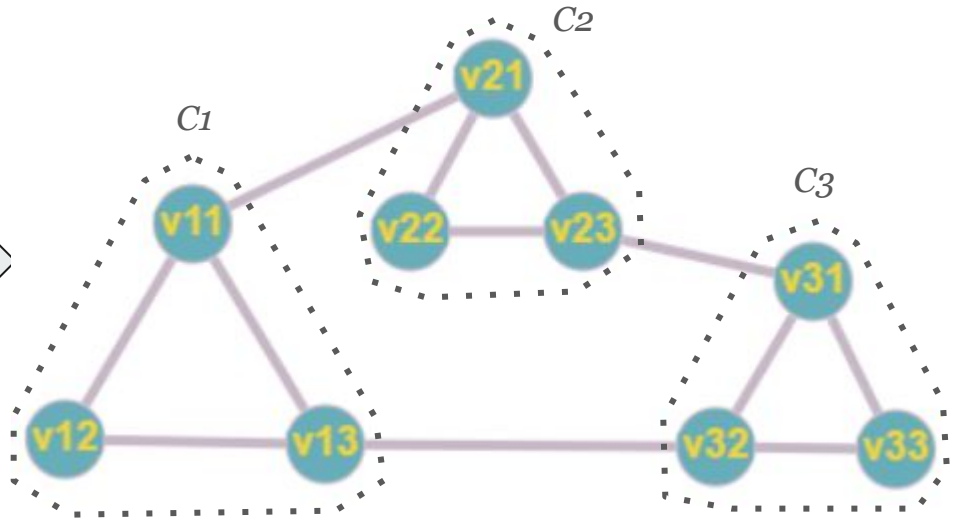
Y como SAT $\leq_P$ 3-SAT, 3-SAT también es *NP-Completo*

Reducciones: Ejemplo 9

Ahora que sabemos que 3-SAT es *NP-Completo*, ¿se podrá reducir 3-SAT a Independent Set? ¿Es Independent Set un problema *NP-Completo*?

Ejemplo:

- $C1=(x_1 \vee x_2 \vee x_3)$
- $C2=(\neg x_1 \vee x_2 \vee x_4)$
- $C3=(\neg x_4 \vee \neg x_3 \vee x_5)$



Reducciones: Ejemplo 9

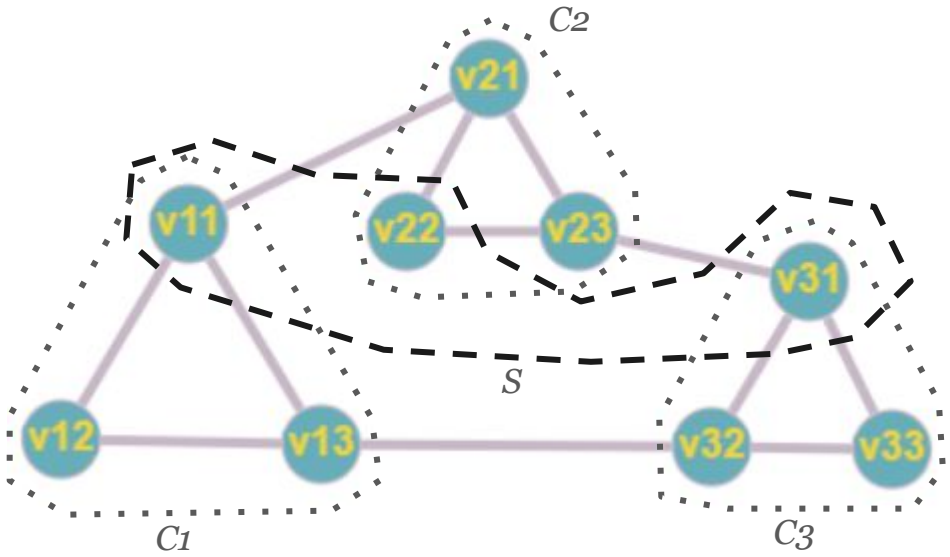
¿Y esto funcionaría? ¿Resolver el Independent Set serviría para resolver el 3-SAT y viceversa? Veamos:
Si 3-SAT se satisface:

- Entonces al menos un término de cada cláusula tiene que ser verdadero, y cada triángulo del grafo contiene un vértice cuyo término asociado vale 1
- Supongamos que existe un conjunto S con un vértice de cada triángulo cuyo término asociado vale 1
- S tiene que ser independiente: si hubiera una arista entre dos vértices de S, tendrían que ser la misma variable de 3-SAT negada, y eso no puede pasar porque 3-SAT se satisface por hipótesis

Reducciones: Ejemplo 9

Ejemplo:

- $C1=(x_1 \vee x_2 \vee x_3)$
- $C2=(\neg x_1 \vee x_2 \vee x_4)$
- $C3=(\neg x_4 \vee \neg x_3 \vee x_5)$



$v=\{x1=1, x2=1, x3=0, x4=0, x5=0\}$

Reducciones: Ejemplo 9

Veamos la vuelta:

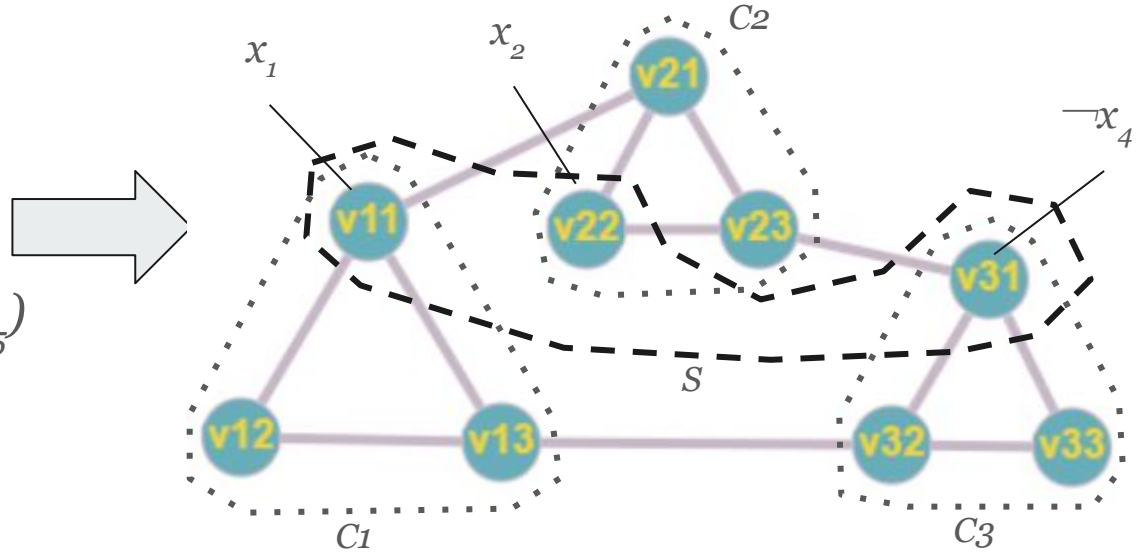
Si el Independent Set S de tamaño k tiene solución:

- Entonces $|S|=k$ y tiene exactamente un vértice de cada triángulo
- Existe una asignación v de 3-SAT tal que todos los vértices de S valen 1, y se construye así:
- Para cada variable x_i de v:
  - Si ninguno de los vértices asociados a x_i o $\neg x_i$ están en S, asignamos $v(x_i)=1$ arbitrariamente
  - Si no, en S está el vértice asociado a x_i o a $\neg x_i$, pero no ambos: de lo contrario existiría una arista entre los dos, contradiciendo el supuesto inicial de que es un Independent Set
    - Si el vértice asociado a x_i es quien está en S, asignamos $v(x_i)=1$
    - Si no, asignamos $v(x_i)=0$

Reducciones: Ejemplo 9

Ejemplo:

- $C1=(x_1 \vee x_2 \vee x_3)$
- $C2=(\neg x_1 \vee x_2 \vee x_4)$
- $C3=(\neg x_4 \vee \neg x_3 \vee x_5)$



Asignación:

$v=\{x1=1, x2=1, x3=1, x4=0, x5=1\}$

Reducciones: Ejemplo 9

3-SAT $\leq_P$ Independent Set

3-SAT	Transformación Entrada	IS	Transform. Salida	3-SAT
Lista inicial de k cláusulas y n variables	• Transformar cada cláusula C_i en un triángulo de vértices v_{ij}, donde v_{ij} representa a la j-ésima variable de la i-ésima cláusula • Unir con aristas cada par de vértices v_{ij} de cada triángulo entre sí • Unir con aristas los vértices v_{ij} de distintos triángulos que representan la misma variable negada	Resolver el Problema de Independent Set de tamaño k	Para cada variable x de 3-SAT • Si tiene vértice asociado en S: • Si está asociado a -x, x=0 • Si está asociado a x, x=1 • Si no, x=1 (arbitr.)	Solución de 3-SAT

$\therefore$ Independent Set es *NP-Completo*

Estrategia para Demostrar que X es NP-Completo

Cuando nos pidan demostrar que X es $NP\text{-}Completo$, debemos:

1. Demostrar que X $\in NP$ (p/ej: proponiendo un certificador en P)
2. Buscar un problema Y $\in NP\text{-}Completo$
3. Probar que Y $\leq_P$ X

Si en el paso 3 usamos una instancia particular S_y de Y para transformarla en una instancia S_x de X, debemos demostrar que:

1. Si S_y es solución de Y, entonces S_x es solución de X
2. Si S_x es solución de X, entonces S_y es solución de Y

Problemas NP-Hard

Si un problema de decisión X está en NP-Completo, entonces el problema de optimización asociado es NP-Hard.



Problemas Co-NP

- Vimos que es relativamente fácil certificar que un problema es NP:
  - Basta con tener una solución y verificar que cumple el problema, en tiempo polinomial.
- ¿Cómo certificamos que un problema NO está en NP?
- Deberíamos probar que NINGUNA solución lo cumple, en tiempo polinomial.



Clase “Co-NP”

- Para cada problema X, existe un problema complementario $\overline{X}$. Para cualquier instancia s, decimos que s pertenece al conjunto de soluciones de $\overline{X}$ si y sólo si s no pertenece al conjunto de soluciones de X.
- Si X está incluido en P, su complemento también. Teniendo un algoritmo que resuelve X, simplemente lo corremos y damos vuelta su respuesta.
- Pero si X está en NP, no podemos hacer lo mismo.
- Si X está en NP, su problema complementario $\overline{X}$ puede estar en NP o no.

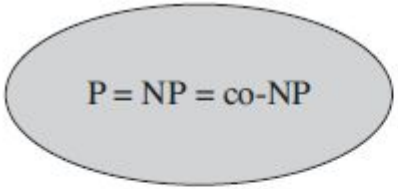
Clase “Co-NP”

Si un problema de decisión X está en NP y su complementario $\overline{X}$ también está en NP, entonces el problema X es un problema $Co\text{-}NP$ (y su complementario también)

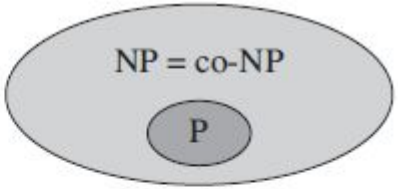
Clase “Co-NP”

- Si X está en NP y también en $Co\text{-}NP$, se dice que es un problema bien caracterizado: Es fácil probar que una instancia tiene solución o que no tiene solución.
- Los problemas en P están en $NP \cap Co\text{-}NP$.
- De hecho, no se conoce ningún problema en $NP \cap Co\text{-}NP$ que no esté en P.
- ¿ $P = NP \cap Co\text{-}NP$?

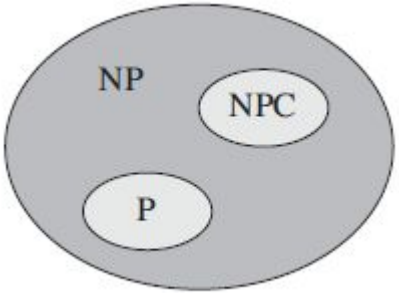
¿Cómo es el mundo?

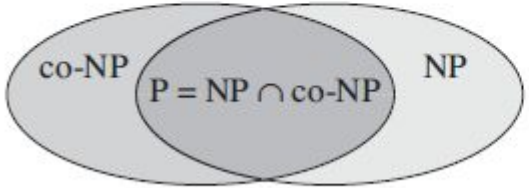


(a)

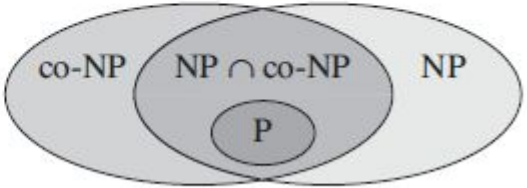


(b)





(c)



(d)

Más allá de la Complejidad



Otras complejidades:

- Dijimos que los recursos limitantes eran tiempo y espacio. ¿Qué pasa con el espacio?
- Clase de problemas $PSPACE$: Necesitan un espacio polinomial para resolverse.
- Clases de problemas EXP: Necesitan tiempo o espacio exponencial para resolverse.
- Clase de problemas R: Problemas que pueden resolverse.
- ¿Problemas que no pueden resolverse?

LA VARA ESTÁ BAJÍSIMA



¡Muchas gracias!